Database Paper: ER Diagram and Explanation

Alvin Ton

CISS 340 Database Concepts

10/24/2025

1 Process

The objective of this Database ER Design is to convert the Employees.xlsx file to an ACID compliant database system. The process to achieve ACID compliance with this database is to follow the 6 steps in designing a data structure (Murach, 2023). The process will be separated into the steps as follows:

1. Identify the data elements & Requirements
2. Subdivide into smallest useful components
3. Identify tables and assign columns
4. Identify primary and foreign keys, and how they are related to each other
5. Review whether the data structure is normalized
6. Identifying the indexes

Further information and justification for each decision regarding each step will be discussed in the respective sections.

1.11 ACID Compliance Definition

ACID is a 4-letter acronym that is defined as atomicity, consistency, isolation, and durability. ACID compliance ensures that databases follow consistent and reliable data operations. To be ACID compliant, these 4 terms need to be integrated into the process of the ER design (Databricks, 2024).

To have atomicity is based on the idea of being all or nothing. When a set of statements is used, it needs to be committed completely. If any part of the statement fails to deliver, then the whole command is rolled back to the previous state; to a point as if the command was never issued to begin with (Databricks, 2024).

Consistency goes over how statements and commands must follow all defined rules, constraints, and adhere to key relationships during, before, and after the statement is issued (Databricks, 2024).

Isolation means that statements that're querying at the same time will not interfere with each other (Databricks, 2024).

Durability talks about how statements that're committed are permanently saved even during cases of outages, restarts, or crashes (Databricks, 2024).

For a database to be ACID compliant, an ER database must be created with proper conventions and rules, with proper normalization reducing redundancy and anomalies when querying or altering data. Proper conventions help create databases that're foundationally trustworthy (Databricks, 2024).

1.1 Identifying Data Elements

To start, the only data element that is relevant to this design is the Excel file Employees.xlsx. The following spreadsheet is formatted with conventional column format for databases left to right, and entries being represented

vertically. Relational databases tend to represent entries horizontally and fields being represented vertically, so that is the first change that will be made (Murach, 2023). The key information showing the spreadsheet is showing employee information of the people working at the company. That is an important scope, because when mentioned later, there are personal information that is not necessary to be kept in the ER design.

Summary: The scope of the design is for employees of a company. Only relevant important information about employees should be kept.

1.2 Subdivide Into Smallest Useful Components

Next, there are some columns that contain information that can be further broken down into smaller, useful components (Murach, 2023). For example, there is a column for the employee name that contains both the first and last name. For better searchability and normalization, the first and last name will be separated

Summary: First name and last name will be separate columns in the ER design.

1.3 Identifying Tables and Columns

In the spreadsheet, all the columns are put in the same table. That introduces unnecessary computational power, as a person querying doesn't need to know all information about an entry at once (Murach, 2023). For this reason, tables should be separated based on the type of information that is relevant for a search. The columns should be separated between a table for employee information, department, job title, personal info, and payroll. The reason for the separation for department information and job title information is to make searches easier and reduce redundancy (Murach, 2023). A department can have many employees, and a job title can belong to many employees. Having the separation makes the relationship clear that an employee has one job and one department, but a department can have multiple employees, and so can a job title. As for the personal info, it is to separate personal information and employment information. Personal information is not needed if the scope of a query is to search for employment information like what is the job title or department of the employee. Only for specific situations will the personal information is needed to specifically be queried. For the payroll table, having a separate table can give better calculations a separation for payment information and employment information. Extra information about the payroll can also be recorded, and act as a form of a record. A payroll table would include a payroll record id to uniquely identify all the payroll entries, an employee id to tie all related payments to an employee, gross pay for payments before deductions, net pay for payments after deductions, a date paid column to show when the payment is due, and pay period column to show the timeframe.

Extra information like emergency contact should be included in the personal information table. Emergency contact information is helpful for emergency situations and the company has a contact to a familiar member or friend . Payroll table is added with extra columns including the payroll record id, employee id, payment period, gross pay, net pay, and date paid (Murach, 2023). The columns for marital status and nationality are removed entirely. These personal information details are not needed, and can potentially be used to profile individuals in a racially discriminative manner.

Summary: Tables are separated into employee, department, job title, personal info, and payroll. Payroll contains new columns including payroll_id, pay period, gross pay, net pay, and date paid. Emergency contact is added to the personal info table, and marital status and nationality are dropped from the database design.

1.4 Identifying Primary and Foreign Key Relationships

All the table separations are all linked to the employee table . A department can have multiple employees, but not the other way around, so the relationship between employee and department is many-to-one (Murach, 2023). That means that the department should keep a primary key to uniquely identify the department, and employee should receive a foreign key tied to the department. Similar situation for a job title; a job title can be shared with multiple

employees, but not the other way around, so the relationship between employee and job title is many-to-one. That means that the job title table should have a primary key to uniquely identify the jobs, and employee should receive a foreign key tied to the job title (Murach, 2023). The payroll table is created as a record that can contain multiple payroll entries tied to the same employee, so the relationship between employee and payment is one-to-many. However, the payroll table needs to still be uniquely identifiable somehow, so the payroll table contains a personal primary key. The payroll is tied to the employee and not directly related to the job title because employees with the same job title can have different salaries based on external factors like performance. For the personal info table, an employee can only have exactly one set of personal information, so the relationship between the employee table and personal info table is one-to-one.

Summary: Employee table has 1 primary key, and 3 foreign keys. The primary key is used to relate to the payroll table, and the personal info table, but also acts as a foreign key for payroll and personal info table. The other 2 foreign keys of the employee table relates to the primary key of job title and department. The department table has 1 primary key, which relates to 1 foreign key belonging to the employee table. The job title contains 1 primary key that relates to 1 foreign key belonging to the employee table. The payroll table contains 1 primary key for unique identification and no relationship, and 1 foreign key that relates to a primary-foreign key of the employee table. The personal info table contains a key that acts as a primary and foreign key to uniquely identify the employee, but also relates to the employee table's primary-foreign key.

1.5 Review Table for Normalization

To satisfy the current ER design, normalization only needs to go up to 3NF. The current relationship design satisfies 1NF where each cell should be a scalar value, and no repeating columns. 2NF is achieved because every non key column depends on the entire primary key for context the foreign keys all rely on the primary key for context, and other on key attributes are tied to the primary key for context as well. Each table has 1 primary key for context, so there are no partial dependencies (Murach, 2023). 3NF is achieved because there are no transitive relationships being created. No non key attributes are depending on another non key attribute but instead relates to a primary key.

Summary: Normalization up to 3NF is checked for and achieved .

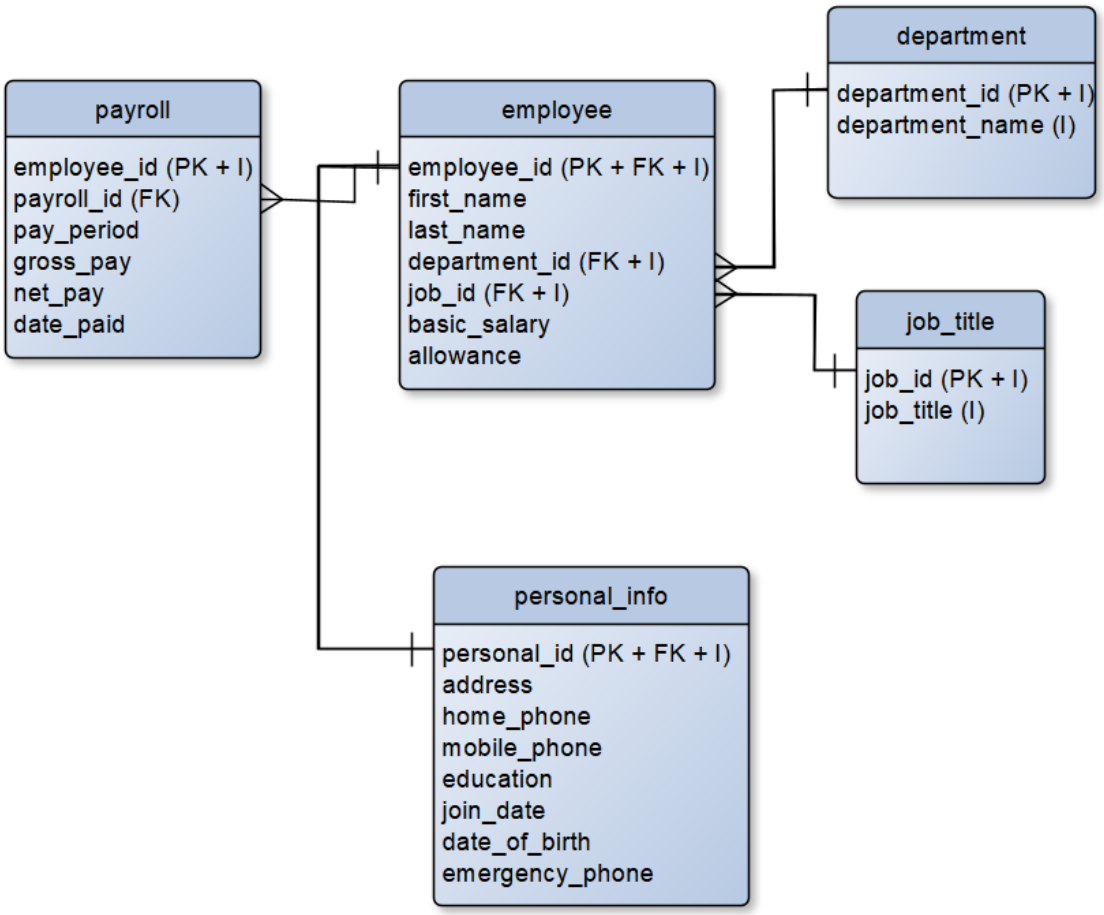
1.6 Identifying Indexes

Indexes should be given to primary keys, unique keys, columns that're queried for often, or not frequently updated (Murach, 2023). That means all primary keys of all 5 tables should be indexed. For the employees table, the foreign keys are used in joins for the department table and job title table, so both should also be indexed separately. The department and job title table contains a unique key for departments and teams, and since joins are going to be used often, between the employee table and department/job title separately, the foreign keys in the employee should be indexed. This adds up to 5 primary (1 each table) + 2 unique (1 for department and job title table)+ 2 foreign keys (both in employee table) = 9 indexes.

Summary: 5 primary keys + 2 unique keys + 2 foreign keys = 9 indexes.

2 Results

The following design shows the ER design for the employee excel spreadsheet using yEd as the modeling. The notation is consistent with standard relationship modeling (GeeksforGeeks, 2016), where a fork denotes a 'many,' and a plus denotes a 'one.' yEd does not have a way to designate certain columns as a primary key, foreign key, or an index, so the notation: PK, FK, and I represented next to the column names in parenthesis are referred to as primary key, foreign key, and index respectively (GeeksforGeeks, 2016).



2.1 Data Dictionary

The following table talks discusses all columns, column data types + key attribute and indices, what table attributes belong to, and a description of the table information, including use, and other modifiers that do not fit under data type or entity. Auto-increment is enabled for each column that is considered a primary key, and null/non-null are chosen based on cell value (Murach, 2023). Foreign keys and primary keys cannot be left null because the columns are related to another key column. Values that are non-key columns are decided to be left nullable if the column is deemed not necessary for any form of information storage. For example, it is possible for the allowance to be 0, and not inherently important to not be null, so the value can be nullable (Murach, 2023). On the flip side, values like first and last name, payroll information, and important personal information are forced to be non-null because those fields are used to either identify the employee, important for emergency contact situations, or is important for a calculation that the value cannot be simply kept null.

Data Item	**Entity**	**Data Type (Format)**	**Description**
employee_id	employee	INT (PK, FK, I)	Unique identifier for each employee, foreign key to personal_info.personal_id, index, non-null, auto-increment

Data Item	Entity	Data Type (Format)	Description
first_name	employee	VARCHAR(50)	Employee's given name, non-unique, non-null, no default.
last_name	employee	VARCHAR (50)	Employee’s surname, non-null, no default.
department_id	employee	INT (FK + I)	Foreign key for JOINs from department table. Non-null, no default.
job_id	employee	INT(FK + I)	Foreign key for JOINs from job_title table. Non-null, no default.
basic_salary	employee	DECIMAL (10, 2)	Basic salary, null, default = 0.
allowance	employee	DECIMAL (10 2)	Allowance, null, default = 0.
department_id	department	INT (PK + I)	Primary for unique identifier, index for JOIN with employee, auto-increment, no null.
department_name	department	VARCHAR(50) (I)	Unique key, name is tied to unique primary key, no-null.
job_id	job_title	INT (PK + I)	Primary for unique identifier, index for JOIN with employee, auto increment, no null.
job_title	job_title	VARCAR(50) (I)	Unique key, index, non-null.
personal_id	personal_info	INT (PK + FK + I)	Unique identifier, primary key, foreign key for one-to-one relationship with employee.employee_id, index, auto-increment, non-null.
address	personal_info	VARCHAR (50)	Address of employee, non-null.

Data Item	Entity	Data Type (Format)	Description
home_phone	personal_info	VARCHAR(50)	Home phone of employee, null, default = NULL
mobile_phone	personal_info	VARCHAR (50)	Mobile phone of employee, non-null
education	personal_info	VARCHAR(50	Education of employee, null, default = NULL
join_date	personal_info	DATE	Join date of employee, non-null
date_of_birth	personal_info	DATE	Date of birth of employee, null, default = NULL
emergency_phone	personal_info	VARCHAR (50)	Emergency contact, non-null.
employee_id	payroll	INT (FK)	Foreign key to employee.employee_id, non-null.
payroll_id	payroll	INT (PK + I)	Unique identifier, primary key, index, auto-increment, non-null
pay_period	payroll	DATE	Pay period for employee salary, non-null.
gross_pay	payroll	VARCHAR(50)	Total pay before deductions for employee, null, default = 0
net_pay	payroll	VARCHAR (50)	Total pay after deductions for employee, null, default = 0
date_paid	payroll	VARCHAR(50	Date employee was paid, Null, default = 0.

3 Conclusion

There were a lot of tricks and gotchas when it came to creating an ER design for the first time. First of all, following the proper steps of creating an ERD is very crucial to make sure that the process doesn't get confusing. The lessons learned here was that working incrementally instead of jumping past steps ends up being more efficient, despite appearing counter-intuitive. Mapping out a plan on what exactly to do for each step saved a lot more time than just brainstorming a bunch of ideas and trying to fit fragmented the ideas together. The most difficult part was accepting the fact that an initial idea of how tables should be structured might not be a plausible solution after thinking about how to maintain normalization. The intuitive initial diagram might be redundant, or potentially even be inefficient despite being non-redundant. The main trick is to think about how to separate data, and deciding what information should be included and what information is just completely unnecessary. For example, ideally in the real world information that can be used to profile people should be removed, and thinking about information like marital status is so inappropriate to keep logged. A company should not need to know whether the person is married or not, or what nationality the person is. Salary is very important and should be a dedicated table in a database to actually show how an employee is getting paid. Another trick is thinking about what the person querying is looking for. A person looking for employment information of an employee doesn't need to know personal identifiable information, so a personal information table is necessary for privacy reasons. Another trick was learning how an ER program works, and choosing the correct notation for the information is important for clarity.

Aspects of this design that could've been done better is formatting the descriptions and thought processes in a way that's easier to understand. Large chunks of paragraphs explaining my thought process isn't easy to read, but there was a degree of depth that I was trying to strive for, and this was the result. Personally, the ER design structure that was chosen in the end admittedly might not be the most efficient, and might end up resulting in more joins and longer commands that could be frustrating to work with. A lot of lessons were learned, and the struggle will actually help in the real world with knowing how important having a defined process is for creating a database.

References

1.) Databricks. (2024). *What are ACID Transactions?* Databricks. https://www.databricks.com/glossary/acid-transactions
2.) GeeksforGeeks. (2016, August 7). *ACID Properties in DBMS*. GeeksforGeeks. https://www.geeksforgeeks.org/dbms/acid-properties-in-dbms
3.) Murach, J. (2023). Murach's MySQL (4th Edition).
4.) yWorks. (2019, January 23). *yEd Live: Entity Relationship Diagram with Martin notation*. YouTube. https://www.youtube.com/watch?v=LNqdh2F0hvY